# Advanced Python

## Topics & Mastery Guide

Syntax · Use Cases · Real Examples

- Decorators
- Generators
- Context Managers
- Metaclasses
- Async/Await
- Descriptors
- Type Hints
- Dataclasses

Generated by Claude · Advanced Python Reference · 2024

# Table of Contents


| # | Topic | Description |
|---|---|---|
| 1 | Decorators | Functions that wrap other functions |
| 2 | Generators & Iterators | Lazy evaluation & memory efficiency |
| 3 | Context Managers | Resource management with 'with' |
| 4 | Async / Await | Concurrent programming |
| 5 | Metaclasses | Classes that create classes |
| 6 | Descriptors | Attribute access control |
| 7 | Type Hints & Annotations | Static type checking |
| 8 | Dataclasses | Boilerplate-free data containers |
| 9 | functools & itertools | Functional programming tools |
| 10 | __slots__ & Memory Tricks | Optimising memory usage |

TOPIC 1 — DECORATORS

# Decorators

A **decorator** is a higher-order function that takes another function as argument, adds behaviour around it, and returns the modified function. They follow the *Open/Closed Principle* — open for extension, closed for modification.

■ **Where to use: Logging, authentication/authorisation, caching, rate-limiting, timing, input validation, retry logic, and API route registration (e.g. Flask @app.route).**

## Syntax

```
def decorator(func):
    def wrapper(*args, **kwargs):
        # --- code before ---
        result = func(*args, **kwargs)
        # --- code after  ---
        return result
    return wrapper

@decorator          # same as:  my_func = decorator(my_func)
def my_func():
    pass
```

### Example 1 — Simple Logger

```
import functools, time

def log_call(func):
    @functools.wraps(func)   # preserves __name__, __doc__
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__} with {args}, {kwargs}')
        result = func(*args, **kwargs)
        print(f'{func.__name__} returned {result}')
        return result
    return wrapper

@log_call
def add(a, b):
    return a + b

add(3, 4)
# Output:
# Calling add with (3, 4), {}
# add returned 7
```

### Example 2 — Decorator with Arguments (Decorator Factory)

```
def repeat(n):             # outer factory
    def decorator(func):   # middle decorator
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
```

```
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f'Hello, {name}!')

greet('Alice')   # prints 3 times
```

### Example 3 — Class-based Decorator

```
class Timer:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func

    def __call__(self, *args, **kwargs):
        start = time.perf_counter()
        result = self.func(*args, **kwargs)
        end   = time.perf_counter()
        print(f'{self.func.__name__} ran in {end-start:.4f}s')
        return result

@Timer
def slow_add(a, b):
    time.sleep(0.5)
    return a + b

slow_add(1, 2)   # slow_add ran in 0.5001s
```

■ ***Note:*** *Always use @functools.wraps(func) inside wrappers to preserve metadata.*

TOPIC 2 — GENERATORS

# Generators & Iterators

Generators are functions that use **yield** instead of **return**. They produce values one at a time (lazy evaluation), consuming O(1) memory regardless of the dataset size. They implement the iterator protocol automatically.

■ **Where to use: Reading huge files line-by-line, infinite sequences, data pipelines, streaming API responses, and paginated database queries.**

## Syntax

```
def generator_function():
    yield value1
    yield value2   # pauses here each time
    # StopIteration raised automatically at end

gen = generator_function()  # returns a generator object
next(gen)                   # get next value
for val in gen:             # iterate all values
    print(val)
```

### Example 1 — Infinite Counter

```
def infinite_counter(start=0, step=1):
    n = start
    while True:
        yield n
        n += step

counter = infinite_counter(10, 2)
print([next(counter) for _ in range(5)])
# [10, 12, 14, 16, 18]
```

### Example 2 — Memory-Efficient File Reader

```
def read_large_file(filepath):
    '''Reads a huge file without loading it all in memory.'''
    with open(filepath) as f:
        for line in f:
            yield line.strip()

# Works on a 10 GB file with minimal RAM
for line in read_large_file('data.csv'):
    process(line)
```

### Example 3 — Generator Expression (one-liner)

```
# List comprehension  → eager, loads all 1M numbers
squares_list = [x**2 for x in range(1_000_000)]

# Generator expression → lazy, O(1) memory
```

```
squares_gen  = (x**2 for x in range(1_000_000))

import sys
print(sys.getsizeof(squares_list))  # ~8 MB
print(sys.getsizeof(squares_gen))   # 104 bytes
```

### Example 4 — send() and Two-Way Communication

```
def accumulator():
    total = 0
    while True:
        value = yield total   # yield sends total, receives value
        if value is None:
            break
        total += value

gen = accumulator()
next(gen)          # prime the generator
gen.send(10)       # total = 10
gen.send(20)       # total = 30
gen.send(5)        # total = 35
```

■ ***Note:*** *yield from delegates to a sub-generator, flattening nested generators cleanly.*

TOPIC 3 — CONTEXT MANAGERS

# Context Managers

Context managers implement the **with** statement. They guarantee setup and teardown code runs even if exceptions occur. Implement via **__enter__**/**__exit__** or **@contextlib.contextmanager**.

■ **Where to use: File I/O, database connections, locks/semaphores, temporary directories, mocking in tests, transaction management, and network sockets.**

## Syntax — Class-Based

```
class ManagedResource:
    def __enter__(self):
        # setup — acquire resource
        return self          # value bound to 'as' variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        # teardown — always runs
        # return True to suppress exceptions
        return False

with ManagedResource() as res:
    res.do_something()
```

### Example 1 — Database Connection Manager

```
import sqlite3

class DatabaseConnection:
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None

    def __enter__(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:             # exception occurred
            self.conn.rollback()  # undo changes
        else:
            self.conn.commit()    # save changes
        self.conn.close()
        return False             # don't suppress exception

with DatabaseConnection('app.db') as conn:
    conn.execute('INSERT INTO users VALUES (?, ?)', (1, 'Alice'))
# Auto-commits on success, rollback on error
```

### Example 2 — Generator-Based (contextlib)

```
from contextlib import contextmanager
import os, shutil, tempfile

@contextmanager
def temp_directory():
    path = tempfile.mkdtemp()
    try:
        yield path              # everything inside 'with' runs here
    finally:
        shutil.rmtree(path)     # always cleanup

with temp_directory() as tmpdir:
    # work with tmpdir — auto-deleted after block
    open(f'{tmpdir}/test.txt', 'w').write('hello')
```

■ ***Note:*** *contextlib.suppress(ExceptionType) is a handy built-in context manager that silences specific exceptions.*

TOPIC 4 — ASYNC / AWAIT

# Async / Await

**asyncio** enables concurrent I/O-bound operations in a single thread using an event loop. **async def** defines a coroutine; **await** suspends it until the awaited operation finishes, allowing other coroutines to run meanwhile.

■ **Where to use: Web scraping, REST API clients, chat servers, database async drivers, file watchers, and any I/O-heavy work where threads would be wasteful.**

## Syntax

```
import asyncio

async def my_coroutine():      # defines a coroutine
    await asyncio.sleep(1)     # suspends, other coroutines run
    return 'done'

# Run top-level
asyncio.run(my_coroutine())
```

### Example 1 — Concurrent HTTP Requests with aiohttp

```
import asyncio
import aiohttp   # pip install aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def fetch_all(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        results = await asyncio.gather(*tasks)  # run concurrently
    return results

urls = ['https://example.com', 'https://httpbin.org/get']
pages = asyncio.run(fetch_all(urls))
# Both requests run at the SAME TIME (not sequentially)
```

### Example 2 — async for & async with

```
import asyncio

class AsyncCounter:
    def __init__(self, limit): self.limit = limit; self.n = 0
    def __aiter__(self): return self
    async def __anext__(self):
        if self.n >= self.limit: raise StopAsyncIteration
        await asyncio.sleep(0.1)   # simulate async work
        self.n += 1
        return self.n
```

```
async def main():
    async for val in AsyncCounter(5):
        print(val)   # 1 2 3 4 5

asyncio.run(main())
```

■ ***Note:*** *Use asyncio.gather() for parallel tasks, asyncio.Queue for producer-consumer patterns.*

TOPIC 5 — METACLASSES

# Metaclasses

In Python, **everything is an object** — including classes. The object that creates a class is called its **metaclass**. By default this is **type**. Customising the metaclass lets you intercept class creation and modify or validate the class before it is used.

■ **Where to use: ORMs (Django models), API frameworks, enforcing coding conventions, singleton patterns, auto-registration of plugins, and interface enforcement.**

## Syntax

```
class MyMeta(type):                        # inherit from type
    def __new__(mcs, name, bases, namespace):
        # called when the class is being CREATED
        cls = super().__new__(mcs, name, bases, namespace)
        return cls

    def __init__(cls, name, bases, namespace):
        # called after __new__
        super().__init__(name, bases, namespace)

class MyClass(metaclass=MyMeta):           # use the metaclass
    pass
```

### Example — Enforcing Abstract Methods Without ABC

```
class InterfaceMeta(type):
    REQUIRED = []
    def __new__(mcs, name, bases, ns):
        cls = super().__new__(mcs, name, bases, ns)
        if bases:   # skip the base class itself
            for method in mcs.REQUIRED:
                if method not in ns:
                    raise TypeError(f'{name} must implement {method}()')
        return cls

class ShapeMeta(InterfaceMeta):
    REQUIRED = ['area', 'perimeter']

class Circle(metaclass=ShapeMeta):
    def area(self): return 3.14 * self.r**2
    def perimeter(self): return 2 * 3.14 * self.r

# class Square(metaclass=ShapeMeta): pass
# → TypeError: Square must implement area()
```

■ ***Note:*** *Prefer ABCs (abc.ABCMeta) for interface enforcement in most cases. Use custom metaclasses only when ABCs are insufficient.*

TOPIC 6 — DESCRIPTORS

# Descriptors

A **descriptor** is any object that defines **__get__**, **__set__**, or **__delete__**. When accessed via a class attribute, Python calls these methods instead of returning the object directly. They power *property*, *classmethod*, *staticmethod*, and ORM field validation.

■ **Where to use: Type-checked attributes, cached properties, lazy loading, ORM column definitions, unit-of-measure conversion, and read-only attributes.**

## Syntax

```
class MyDescriptor:
    def __set_name__(self, owner, name):          # called at class creation
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None: return self               # accessed on class
        return obj.__dict__.get(self.name)

    def __set__(self, obj, value):
        obj.__dict__[self.name] = value

    def __delete__(self, obj):
        del obj.__dict__[self.name]
```

### Example — Typed / Validated Attribute

```
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.private = f'_{name}'

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return getattr(obj, self.private, None)

    def __set__(self, obj, value):
        if not isinstance(value, (int, float)):
            raise TypeError(f'Expected number, got {type(value)}')
        if value <= 0:
            raise ValueError(f'Must be positive, got {value}')
        setattr(obj, self.private, value)

class Product:
    price    = PositiveNumber()
    quantity = PositiveNumber()

p = Product()
p.price = 9.99    # OK
p.price = -5      # ValueError: Must be positive
```

■ ***Note:*** *Python's built-in @property is syntactic sugar for a simple descriptor.*

TOPIC 7 — TYPE HINTS

# Type Hints & Annotations

Type hints (PEP 484) let you annotate variables, parameters, and return values with expected types. Python does **not** enforce them at runtime — they are checked by tools like **mypy**, **pyright**, and IDEs. They vastly improve readability, refactoring safety, and auto-completion.

■ **Where to use: Any codebase with more than one developer, library APIs, long-lived projects, and anywhere IDE support or static analysis matters.**

## Syntax

```
# Variables
name: str = 'Alice'
count: int = 0
ratio: float = 0.5
flag: bool = True

# Functions
def greet(name: str, times: int = 1) -> str:
    return (f'Hello {name}! ' * times).strip()

# Collections
from typing import List, Dict, Tuple, Optional, Union
def process(items: List[int]) -> Dict[str, int]: ...

# Python 3.10+ shorthand
def foo(x: int | None) -> list[str]: ...
```

### Example — Full Annotated Class

```
from __future__ import annotations
from typing import Optional, ClassVar

class BankAccount:
    bank_name: ClassVar[str] = 'PyBank'

    def __init__(self, owner: str, balance: float = 0.0) -> None:
        self.owner: str = owner
        self.balance: float = balance
        self._history: list[float] = []

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError('Amount must be positive')
        self.balance += amount
        self._history.append(amount)

    def get_history(self) -> list[float]:
        return self._history.copy()

    @classmethod
```

```
    def from_dict(cls, data: dict[str, object]) -> BankAccount:
        return cls(str(data['owner']), float(data['balance']))
```

■ ***Note:*** *Run: mypy your_file.py to check type errors statically before running code.*

TOPIC 8 — DATACLASSES

# Dataclasses

**@dataclass** (PEP 557) auto-generates **__init__**, **__repr__**, and **__eq__** from class-level annotations. They drastically reduce boilerplate for data-holding classes, with options for ordering, freezing (immutability), and post-init processing.

■ **Where to use: Configuration objects, DTOs, API request/response models, value objects, command-pattern payloads, and anywhere you store structured data.**

## Syntax

```
from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float
    z: float = 0.0          # default value

p = Point(1.0, 2.0)
print(p)   # Point(x=1.0, y=2.0, z=0.0)   <- auto __repr__
p == Point(1.0, 2.0)       # True           <- auto __eq__
```

### Example — Full-Featured Dataclass

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass(order=True, frozen=True)   # orderable + immutable
class Employee:
    sort_index: float = field(init=False, repr=False)  # computed
    name:       str
    department: str
    salary:     float
    skills:     list[str] = field(default_factory=list)
    joined:     datetime  = field(default_factory=datetime.now)

    def __post_init__(self):
        # runs after __init__ — use object.__setattr__ when frozen
        object.__setattr__(self, 'sort_index', self.salary)

    @property
    def annual(self) -> float:
        return self.salary * 12

e1 = Employee('Alice', 'Eng', 5000, ['Python', 'SQL'])
e2 = Employee('Bob',   'Eng', 6000)
print(e1 < e2)    # True — sorted by salary (sort_index)
# e1.salary = 9999   # FrozenInstanceError — immutable!
```

■ ***Note:*** *For complex validation and serialisation, consider Pydantic which extends dataclass ideas with runtime type enforcement.*

TOPIC 9 — FUNCTOOLS & ITERTOOLS

# functools & itertools

These standard-library modules provide powerful functional-programming and iterator utilities. Mastering them allows you to write expressive, memory-efficient pipelines with minimal code.

## functools highlights

```
import functools

# --- lru_cache: memoisation ---
@functools.lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

# --- partial: fix arguments ---
from functools import partial
power_of_2 = partial(pow, 2)   # pow(2, ?)
print(power_of_2(10))          # 1024

# --- reduce: fold a sequence ---
from functools import reduce
product = reduce(lambda a, b: a * b, [1, 2, 3, 4, 5])
# product = 120

# --- singledispatch: function overloading ---
@functools.singledispatch
def process(val):
    raise TypeError(f'No handler for {type(val)}')

@process.register(int)
def _(val): return val * 2

@process.register(str)
def _(val): return val.upper()

process(5)      # 10
process('hi')   # 'HI'
```

## itertools highlights

```
import itertools

# chain: concatenate iterables lazily
list(itertools.chain([1,2], [3,4], [5]))   # [1,2,3,4,5]

# product: Cartesian product
list(itertools.product('AB', repeat=2))    # AA AB BA BB

# groupby: group consecutive elements
data = sorted([('a',1),('b',2),('a',3)], key=lambda x: x[0])
```

```
for key, group in itertools.groupby(data, key=lambda x: x[0]):
    print(key, list(group))

# islice: lazy slice (no index support needed)
first_5 = list(itertools.islice(some_generator(), 5))

# accumulate: running totals
import operator
list(itertools.accumulate([1,2,3,4,5], operator.mul))
# [1, 2, 6, 24, 120]  — running product
```

TOPIC 10 — __slots__ & MEMORY

# __slots__ & Memory Optimisation

By default Python stores each instance's attributes in a __**dict**__ (a hash-map), which adds ~200–300 bytes per object. Defining __**slots**__ replaces the per-instance dict with fixed-size slots, cutting memory usage by 30–50% and speeding up attribute access.

■ **Where to use: Classes instantiated millions of times (network packets, game entities, ML data-point objects, financial tick data).**

```
import sys

class WithDict:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class WithSlots:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

d = WithDict(1, 2)
s = WithSlots(1, 2)

print(sys.getsizeof(d))   # ~48 bytes + __dict__ ~232 bytes = ~280
print(sys.getsizeof(s))   # ~56 bytes — NO __dict__

# Restriction: cannot add new attributes at runtime
# d.z = 3    # OK
# s.z = 3    # AttributeError
```

## Combining __slots__ with inheritance

```
class Base:
    __slots__ = ('id',)

class Child(Base):
    __slots__ = ('name', 'value')  # extend, don't repeat 'id'

c = Child()
c.id = 1; c.name = 'x'; c.value = 42  # all three work
```

■ ***Note:*** *__slots__ cannot be used with multiple inheritance when both bases define different __slots__ — use with care.*

# Quick Reference — All Topics

| Topic | Key Syntax | Primary Use Case |
|---|---|---|
| Decorators | `@decorator / @decorator(args)` | Logging, caching, auth |
| Generators | `yield / yield from` | Lazy sequences, streaming |
| Context Managers | `__enter__ / __exit__ / @contextmanager` | Resource cleanup |
| Async/Await | `async def / await / asyncio.run()` | Concurrent I/O |
| Metaclasses | `class Meta(type): __new__` | ORM, plugin registry |
| Descriptors | `__get__ / __set__ / __delete__` | Validated attributes |
| Type Hints | `var: Type / def f() -> T` | Static analysis, docs |
| Dataclasses | `@dataclass(frozen, order)` | Data containers |
| functools | `lru_cache, partial, reduce` | Functional utilities |
| __slots__ | `__slots__ = ('x', 'y')` | Memory optimisation |

*© Advanced Python Topics Guide · Created with Claude · All examples are original and runnable on Python 3.9+*